TECHNISCHE HOCHSCHULE MITTELHESSEN | CAMPUS FRIEDBERG | MND Mathematik, Naturwissenschaften und Datenverarbeitung

Skript

Conflict-free Replicated Datatypes

im Rahmen des Moduls

PI5002 – Fortgeschrittene Datenbanktechnologien

Verfasser: Steven Johnston
Abgabe: 05.06.2023

Inhaltsverzeichnis

1. Motivation 3
2. Textbearbeitung 4
3. Einführung in CRDTs 7
4. Arten von CRDTs 8
5. Funktionsweise von CRDTs 9
6. Konvergenz 10
7. Verschachtelungen 12
8. Anwendung von CRDT in Listen 15
9. CRDTs in Baumstrukturen 19
10. Performance von CRDT-Algorithmen 22
11. Zusammenfassung 23
Quellen 24

1. Motivation

Conflict-free Replicated Datatypes (CRDT) ist eine Datenstruktur, die eine Lösung für die Synchronisation von Daten in verteilten Umgebungen bereitstellt.[1] Die Replikation und die Konsistenz bilden grundlegende Merkmale jedes großen verteilten Systems. Dazu gehören das World Wide Web, Peer-to-Peer- oder Cloud-Computing-Plattformen.[2] In verteilten Umgebungen haben Benutzer nicht zu jedem Zeitpunkt eine aktive Systemverbindung, weshalb durchgeführte Operationen vom System nicht unmittelbar in die Umgebung übernommen werden.

Der Ausdruck „conflict-free“ bedeutet in diesem Zusammenhang, dass die Datenstruktur keine exklusiven Schreibrechte für Operationen erfordert. Darüber hinaus können gleichzeitige Aktualisierungen erkannt und Konflikte automatisch gelöst werden, wobei der Ausdruck „gleichzeitig“ als das Eintreten zweier Ereignisse ohne Kenntnis voneinander definiert wird. Hierbei werden Aktualisierungen in einer bestimmten Reihenfolge verarbeitet, damit jeder Benutzer final das gleiche Ergebnis erhält. Zu beachten ist, dass Konflikte weiterhin auftreten können, CRDT jedoch Vorhersagen zum Lösen dieser Konflikte bereitstellt. Dies geschieht durch das Lesen von Metadaten der Datenstruktur.[3]

Einige Use Cases und Implementierungen, die CRDTs nutzen, sind die Folgenden:

- Amazon verwendet CRDTs, um Warenkörbe zu synchronisieren
- Riat Games nutzt CRDTs für den In-Game-Chat seines Spiels League of Legends
- Rovio (Angry Birds), SoundClound, TomTom und CREAustralia verwendet CRDTs für deren Clickstream-Analysen[4]

CRDT ermöglicht es mehreren Knoten, gleichzeitige Änderungen an einem Dokument vorzunehmen, während der implementierte Algorithmus der Zusammenführung dieser Aktualisierungen dient. Der Einsatz von CRDT ist vielfältig, sodass Datentypen wie Mengen, Listen, Key-Value-Maps, Diagramme, Counter und JSON-Bäume mit CRDT verarbeitet werden können. Ein wichtiges Merkmal der CRDT ist hierbei die Fähigkeit

[1] https://www.bartoszsypytkowski.com/the-state-of-a-state-based-crdts/
[2] https://pages.lip6.fr/Marek.Zawirski/papers/RR-7687.pdf, S. 3
[3] https://www.bartoszsypytkowski.com/the-state-of-a-state-based-crdts/
[4] https://www.bartoszsypytkowski.com/the-state-of-a-state-based-crdts/

der Konvergenz durch Kommutativität. Diese Fähigkeit gewährleistet, dass die Zustände des Dokuments am Ende der Aktualisierung identisch sind, ungeachtet der Reihenfolge der Änderungen oder der Konnektivität der Benutzer. Hierfür nutzt CRDT zusätzliche Metadaten. Die Algorithmen fügen jeder Operation (einfügen oder löschen) eine eindeutige Kennung zu – in den meisten Fällen eine Zahl. Diese ID wird im Nachgang verwendet, um auf bestimmte Teile der Datenstruktur zu verweisen.[5] Dadurch werden die Operationen bei gleichzeitigen Aktualisierungen eindeutig identifiziert und am Ende sinnvoll zusammengeführt werden. Auf diese Weise können verschiedene Replikate ihren Zustand zusammenführen und zu einem einheitlichen Ergebnis konvergieren. Dies ermöglicht eine effiziente Synchronisation von Daten in verteilten Systemen und gewährleistet, dass alle Knoten die gleichen Informationen enthalten, unabhängig davon, wann und in welcher Reihenfolge die Aktualisierungen erfolgt sind.[6]

2. Textbearbeitung

In Kollaborationssoftware (Google Docs, Office 365 etc.) besteht die Möglichkeit des gemeinsamen Arbeitens mit mehreren Benutzern. Textdokumente oder Tabellenkalkulationen können gleichzeitig und dezentral aktualisiert werden. Beim Anzeigen eines Dokuments wird eine Kopie auf den Computer des Benutzers geladen. Dies erfolgt in einem Webbrowser oder in einer nativen App auf dem lokalen Gerät. Sämtliche Änderungen werden auf die lokale Kopie des Dokuments angewendet und asynchron an alle anderen Benutzer, die ebenfalls eine Kopie des Dokuments besitzen, übertragen. Im Beispiel von Google Docs läuft dieser Austausch von Änderungen über einen bereitgestellten Server von Google. Dieses Szenario der Zusammenarbeit ähnelt der Replikation in verteilten Datenbanken und es können ähnliche Probleme auftreten, wie es bei der kollaborativen Textbearbeitung der Fall ist. Die gemeinsam genutzten Daten stellen in diesem Kontext eine Datenbank dar. Innerhalb dieser wird jeder Knoten, der eine Kopie der Daten besitzt, als Replikat bezeichnet.[7]

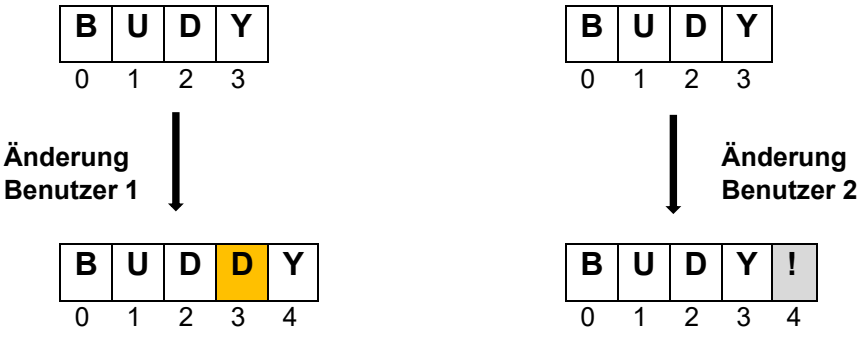
---

[5] https://martin.kleppmann.com/papers/convergence-cacm.pdf, S. 105
[6] https://martin.kleppmann.com/papers/convergence-cacm.pdf, S. 105
[7] https://martin.kleppmann.com/papers/interleaving-papoc19.pdf

Ziel ist es, dass alle Benutzer, die das Dokument gleichzeitig aktualisieren, denselben Zustand erreichen. Die Änderungen im Textdokument müssen durch die Algorithmen dementsprechend vorgenommen werden, dass sie Sinn ergeben. Nachfolgend werden zwei Algorithmen (Algorithmen für Konvergenz) vorgestellt, die zu diesem Zielzustand führen.

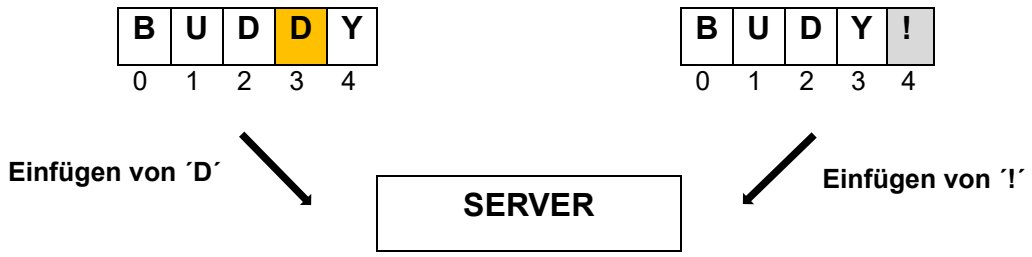
Der erste Algorithmus ist die Operationale Transformation (OT). Dieser kommt oftmals in Echtzeit-Editoren wie Google Docs zum Einsatz. Der Text wird als lineare Folge von Zeichen strukturiert. Durch das Einfügen oder Löschen von Zeichen kann die Datenstruktur an jeder beliebigen Position aktualisiert werden. Dieser Algorithmus ordnet gleichzeitige Operationen durch restriktive Regeln neu zu und arbeitet demnach allgemeiner als die Kommutativität, die von CRDTs verwendet wird. Heute wird die OT als eher klassisch angesehen und wurde bis ca. 2006 verwendet.[8]

Die OT funktioniert in der Form, dass Indexe die Position der Zeichen im Dokument beschreiben. Ändern zwei Benutzer das Dokument durch Einfügen von Zeichen, z.B. Benutzer 1 fügt ein weiteres ´D´ an Position 3 ein und Benutzer 2 fügt ein ´!´ an Position 4 ein, sieht dies wie folgt aus:



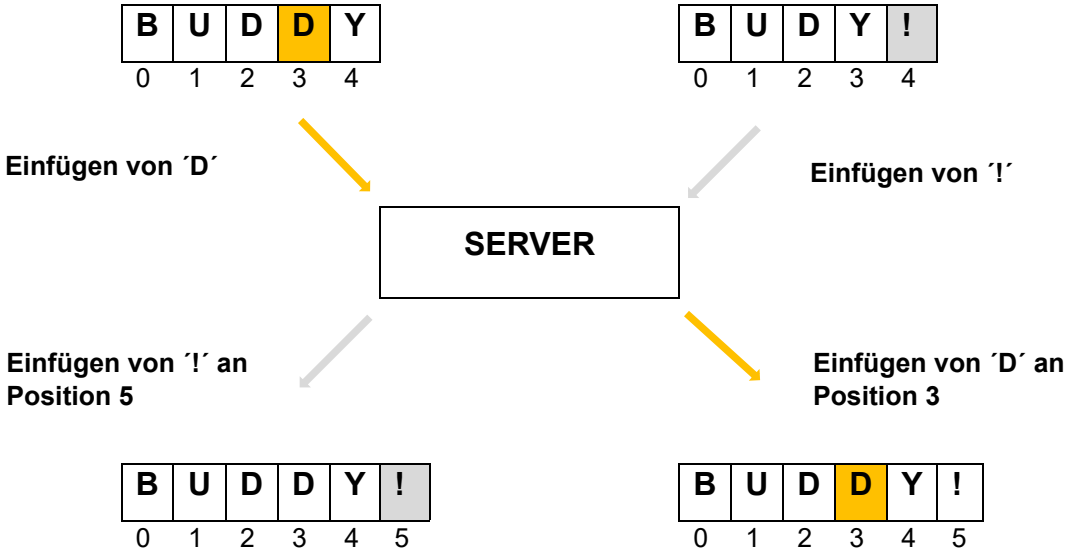
[8] https://dl.acm.org/doi/pdf/10.1145/3392825, S. 1

Im Anschluss werden diese Änderungen von einem Server kommuniziert und die Benutzer informiert:



Quelle: in Anlehnung: https://dl.acm.org/doi/pdf/10.1145/3392825, S. 5

Die Änderung wird erst im nächsten Schritt vom Server verarbeitet und der rechten Seite zugeschrieben. Der Server fügt die Änderung ´D´ an Position 3 ein und bildet das Produkt ´BUDDY!´ – ein sinnvolles Konstrukt. Schreibt der Server jedoch die rechte Seite zuerst auf die linke Seite, erhalten wir das Produkt ´BUDD!Y´ und enden mit zwei Benutzern, die jeweils eine andere Version des Dokuments besitzen (an dieser Stelle nicht abgebildet). Nachfolgend wird deutlich, dass als Voraussetzung zunächst die Transformation der Indexe zu erfolgen hat. In diesem Beispiel das ´!´ von Index 4 auf Index 5:[9]



Quelle: in Anlehnung: https://dl.acm.org/doi/pdf/10.1145/3392825, S. 5

[9] https://dl.acm.org/doi/pdf/10.1145/3392825, S. 4

Die Grundannahme bei der OT ist, dass stets ein zentraler Server existiert, der die Abfolge der Vorgänge bestimmt. Der Server ist daher als Schlüsselrolle zu betrachten, da es keine weiteren Kommunikationskanäle geben kann. Der Unterschied zu CRDT besteht darin, dass die Datenstruktur eine Multi-User-Kollaboration ermöglicht, ohne Annahmen über Netzwerk Richtlinien vorauszusetzen. Die Übermittlungen von Operationen zwischen zwei Benutzern können über jede Art von Kommunikationskanal stattfinden.[10]
Zusammenfassend lässt sich sagen, dass OT die Position von eingefügten oder gelöschten Zeichen durch das Festlegen eines Index bestimmt. Dieser wird ab Beginn des Dokuments gezählt und fordert daher eine Transformation. Der im Nachgang vorgestellte zweite Ansatz CRDT vermeidet diese operative Transformation der Indexe.

3. Einführung in CRDTs

Es gibt verschiedene Systeme, welche Kopien von Daten auf mehreren Computergeräten speichern müssen. Mobile Anwendungen, verteilte Datenbanken und Kollaborationssoftware können hier als Beispiel für solche Systeme genannt werden. Anwendungsfälle sind Synchronisationsoperationen von Kalenderdaten, Notizen und Kontakte auf lokalen Geräten eines Benutzers. Datenbanken nutzen unter anderem verschiedene Replikate, um zu gewährleisten, dass Daten permanent verfügbar sind. Als Replikate sind in diesem Kontext die Kopien der Daten zu verstehen. Die Kollaborationssoftware hat zu gewährleisten, dass gleichzeitige Änderungen von unterschiedlichen Benutzern an einem Dokument möglich sind.[11]
Es gibt zwei Ansätze zur Verwaltung von Änderungen eines Dokuments. Der erste Ansatz ist die „Strenge konsistente Replikation“ (strong consistency replication), welche eine zeitgleiche Änderung der Daten durch das System nicht zulässt. Die Replikate entscheiden, wann und in welcher Form eine Änderung vorgenommen werden soll. Die Berechnungen der Koordinierung geht zu Lasten der Latenzzeit. Der zweite Ansatz ist die „Optimistische Replikation (optimistic replication) und ermöglicht die parallele Bearbeitung eines Dokuments durch mehrere Benutzer.

[10] https://dl.acm.org/doi/pdf/10.1145/3392825, S. 7-8
[11] https://crdt.tech/

Im Fall der optimistischen Replikation können die Zustände der Dokumente voneinander abweichen.[12] Es ist demnach abschließend notwendig, alle Änderungen zu einem konsistenten Dokument zusammenzuführen. Dies gewährleistet, dass keine Änderungen verloren gehen. Bei diesem Ansatz können die Benutzer die Daten auf jedem Replikat unabhängig der Kommunikation ändern. Konflikte entstehen jedoch, wenn mehrere Benutzer gleichzeitig Daten im Dokument ändern. CRDT ermöglicht ein automatisiertes Zusammenführen der Änderungen und wird in Systemen mit optimistischer Replikation angewandt, um auftretende Konflikte zu beheben.[13]
Der Text eines Dokuments wird z.B. als Liste von Zeichen dargestellt. Unabhängig davon, ob etwas gelöscht oder eingefügt wird, erhält jede Änderung am Dokument eine ID. Diese lokalen Änderungen werden an die Replikate weitergeben. CRDTs können außerdem im dezentralen Betrieb eingesetzt werden, da sie keinen zentralen Server voraussetzen. [14]

4. Arten von CRDTs

Es gibt zwei Arten von CRDTs, die anhand zwei wesentlicher Merkmale unterschieden werden:

- State-Based CRDTs
  Hier handelt es sich um zustandsbasierte (konvergente) Datentypen – früher auch als `Convergent Replicated Data Types` (CvRDTs) bezeichnet.
- Operation-Based CRDTs
  In diesem Fall wird von operationsbasierten (kommutativen) Datentypen gesprochen – früher auch als `Commutative Replicated Data Type` (CmRDT) bezeichnet.

Beide CRDTs bestehen aus dem Replikationsprotokoll und dem Algorithmus für die Zustandsanwendung. Die Unterscheidung und damit die wesentlichen Merkmale der beiden Arten liegt in der Implementierung und deren Schwerpunkte. Zur Anwendung einer automatisierten Konfliktlösung werden Metadaten benötigt. Bei State-Based

[12] https://crdt.tech/
[13] https://crdt.tech/
[14] https://martin.kleppmann.com/papers/interleaving-papoc19.pdf

CRDTs sind diese als Teil der Datenstruktur vorhanden, wohingegen sie bei Op-Based CRDTs im Replikationsprotokoll enthalten sind.[15]

State-Based CRDTs

Die Funktionsweise der State-Based CRDTs ist die, dass die Replikate ihren gesamten Zustand über das Netzwerk an andere Replikate senden. Erhält ein Replikat einen neuen Zustand (state), wird dieser durch die Merge-Methode im empfangenden Replikat zusammengeführt.[16] Die Merge-Methode gilt als wichtige Operation, um Merge-Konflikte aufzulösen. Das Ziel ist es, zwei korrespondierende Replikate auf einen einheitlichen aktualisierten Zustand zu bringen. Die Merge-Operation bietet durch ihre drei Eigenschaften die folgenden Vorteile: Die **(1)** Kommutativität und **(2)** Assoziativität stellen sicher, dass Merge-Operationen ungeachtet der Reihenfolge bearbeitet werden können und das Resultat dennoch den korrekten Zielzustand liefert. Die **(3)** Idempotenz gewährleistet durch die Replikationsschicht die Löschung von Duplikaten und verhindert somit ihre Präsenz im Resultat.[17]

Operation-based CRDT:

Die zweite Art der CRDTs ist die Operation-Based CRDT, bei dem jede Änderung der replizierten Daten als Vorgang kodiert wird. Im Gegensatz zur State-Based CRDT werden in diesem Fall Objekte (Tupel) repliziert und die Operationen über das Netz gesendet.[18] Dieser Vorgang hat sicherzustellen, dass die Replikate bei gleichzeitigen Operationen zusammengeführt werden können. Die Reihenfolge der Änderungen spielen dabei keine Rolle.[19]

5. Funktionsweise von CRDTs

CRDTs vergeben für jedes einzelne Zeichen im Dokument eine eindeutige ID. Diese ID ist dauerhaft der Position im Dokument zugeschrieben, selbst wenn Operationen stattfinden. Eine Möglichkeit sie zu bestimmen ist die Berechnung mit Hilfe von rationalen Zahlen zwischen 0 und 1, wobei 0 den Beginn und 1 das Ende des Dokuments darstellt. Positionen werden ermittelt, indem eine Zahl innerhalb der

[15] https://www.bartoszsypytkowski.com/the-state-of-a-state-based-crdts/
[16] https://crdt.tech/glossary
[17] https://www.bartoszsypytkowski.com/the-state-of-a-state-based-crdts/
[18] https://pages.lip6.fr/Marek.Zawirski/papers/RR-7687.pdf, S. 7
[19] https://crdt.tech/glossary

Zahlenreihe von 0 bis 1 ausgewählt wird. Im Beispiel ´Budy´ erhält das ´B´ die Zahl 0.2, das ´U´ 0.4, das ´D´ 0.6 und das ´Y´ 0.8. Möchte ein Benutzer eine Operation ausführen und ein weiteres ´D´ an vierter Stelle einfügen, so kann der Mittelwert der angrenzenden Stellen gebildet werden, wodurch das neue ´D´ den Wert 0.7 annimmt. Das Textdokument wird als eine Art Tupel angesehen, welches den Zahlenwert der Position und eine ID des Benutzers enthält. In nachfolgendem Beispiel stehen `M` und `N` jeweils für den Benutzer.[20] Daraus lässt sich folgendes bauen:

B	U	D	Y
0.2	0.4	0.6	0.8

= {(0.2, M, ´B´), (0.4, M, ´U´), (0.6, M, ´D´), (0.8, M, ´Y´)}

Änderung = {(0.7, N, ´D´)}

B	U	D	D	Y
0.2	0.4	0.6	0.7	0.8

= {(0.2, M, ´B´), (0.4, M, ´U´), (0.6, M, ´D´), (0.7, N, ´D´), (0.8, M, ´Y´)}

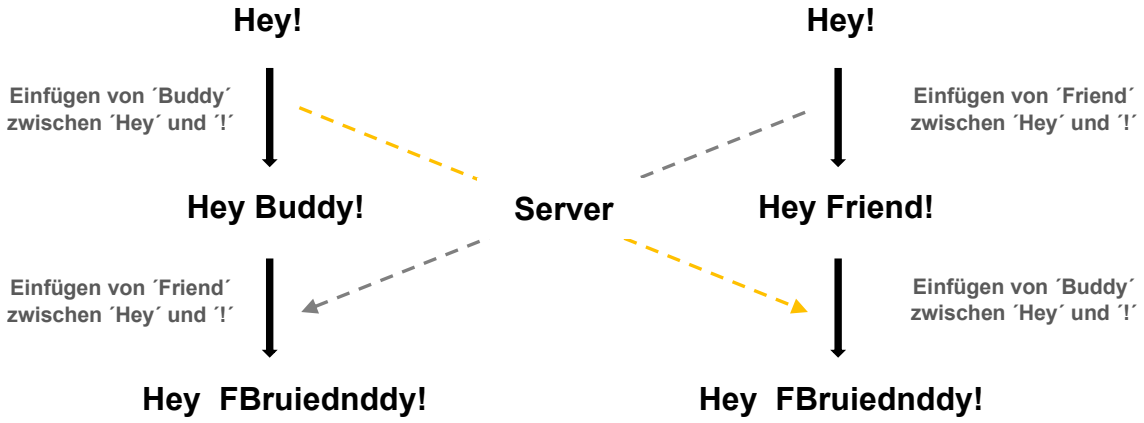
Diese Vorgehensweise ist bei kollaborativem Arbeiten notwendig, da zwei oder mehr Benutzer die Möglichkeit haben, die gleiche Position zum gleichen Zeitpunkt im Dokument zu ändern. Die Knoten-ID dient hierbei als Grundlage für die Reihenfolge der vorgenommenen Änderungen an den Positionen und die Tupel repräsentieren das Dokument. Die Tupel ermöglichen es, Änderungen vorzunehmen, neue Zeichen einzufügen oder zu löschen. Der Zustand des Dokuments wird durch die Sortierung der Zahlen exakt strukturiert.[21]

6. Konvergenz

Die Eigenschaft der Konvergenz, die die CRDT-Algorithmen aufweisen, garantiert die Korrektheit der Algorithmen und bedeutet, dass sich zwei Replikate im gleichen Zustand befinden, selbst wenn Aktualisierungen in unterschiedlicher Reihenfolge stattfinden. Die Konvergenzeigenschaft stellt die automatisierte Zusammenführung der konkurrierenden Operationen sicher. Die Operationen sind kommutativ und gewährleisten so, dass beide Replikate denselben Endzustand erhalten.[22] Für die

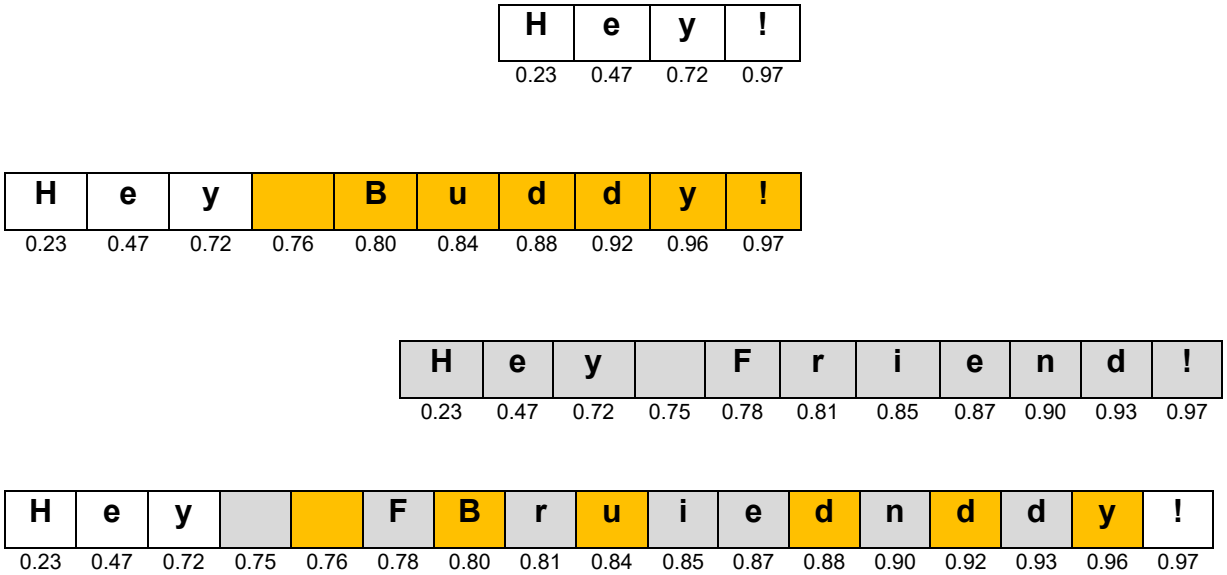
[20] https://dl.acm.org/doi/pdf/10.1145/3301419.3323972, S. 3
[21] https://dl.acm.org/doi/pdf/10.1145/3301419.3323972, S. 1
[22] https://martin.kleppmann.com/papers/interleaving-papoc19.pdf, S.3

korrekte Verarbeitung des Algorithmus ist der Endzustand von erheblicher Bedeutung. Der Algorithmus führt Operationen zusammen und liefert einen Endzustand. Hierbei ist es schwierig zu definieren, ob der Endzustand wünschenswert ist. Diese Einschätzung liegt im menschlichen Ermessen. Es ist festzuhalten, dass Konvergenz eine Eigenschaft ist, die sicherstellt, dass nach jeder Kommunikation alle Benutzer denselben Endzustand vorliegen haben. Eine Gewährleistung von sinnvollen Ergebnissen kann durch die Konvergenz jedoch nicht gegeben werden. Zur Verdeutlichung nachfolgend ein Beispiel einer gleichzeitigen Operation.



Quelle: https://martin.kleppmann.com/papers/interleaving-papoc19.pdf, S.2

Sobald der Server die Operationen kommuniziert, verschachtelt der Algorithmus die beiden Einfügungen Zeichen für Zeichen. Der Algorithmus führt die Rechnungen richtig aus, der Endzustand ist nach menschlichem Ermessen jedoch nicht wünschenswert.



Quelle: https://martin.kleppmann.com/papers/interleaving-papoc19.pdf, S.2

7. Verschachtelungen

Das zuvor beschriebene Verhalten der Verschachtelung (engl. Interleaving) kann bei vielen Algorithmen beobachtet werden. Verschachtelungen sind im Algorithmus fest implementiert und treten je nach Design in unterschiedlicher Weise auf. Es gibt Algorithmen, die nur dann eine Verschachtelung aufweisen, wenn mehrere Replikate gleichzeitig Änderungen vornehmen. Ein Beispiel hierfür ist ein Benutzer, der während der Bearbeitung sein Endgerät wechselt sowie ein zweiter Benutzer, der offline über ein drittes Endgerät Änderungen vornimmt.[23]

In Tabelle 1 sind verschiedene Algorithmen aufgeführt, die dahingehend bewertet wurden, ob sie die unerwünschte Verschachtelung ausschließen. Lediglich der Algorithmus `Fugue` ist mit dem Ergebnis `Keine Verschachtelung` bewertet. Es ist jedoch festzuhalten, dass keiner der untersuchten Algorithmen Verschachtelungen vollständig ausschließt.

Textbasierte CRDTs	
Logoot	Verschachtelung
LSEQ	Verschachtelung
WOOT	Verschachtelung
Treedoc	Verschachtelung
RGA	Weniger Anomalie
Yjs	Weniger Anomalie
Fugue	Keine Verschachtelung

Tabelle 1. Quelle: https://arxiv.org/pdf/2305.00583.pdf, S. 5

LSEQ hat das Problem der Verschachtelung, weil die Designer einen effizienten Algorithmus konstruieren wollten, der weniger Speicherkapazität für die IDs verwendet. In dieser Hinsicht ist LSEQ im Gegensatz zu WOOT zwar effizienter, diese Leistungsoptimierung führt jedoch zu einem schlechteren Verhalten des Algorithmus.

RGA weist ein geringeres Verschachtelungsproblem auf. Die Abbildung 1 zeigt, dass User 1 das Dokument mit dem Wort `Hello!` beginnt und danach `reader` einfügt. Anschließend bewegt User 1 den Cursor nach dem Wort `Hello` zurück und fügt `dear` ein. Nun betrachten wir User 2, welcher das Wort `Alice` einfügt. Bei RGA kann dies

[23] https://arxiv.org/pdf/2305.00583.pdf, S. 5

dazu führen, dass beide Operationen ein wenig durcheinandergeraten, jedoch nicht Zeichen für Zeichen verschachtelt werden.[24]

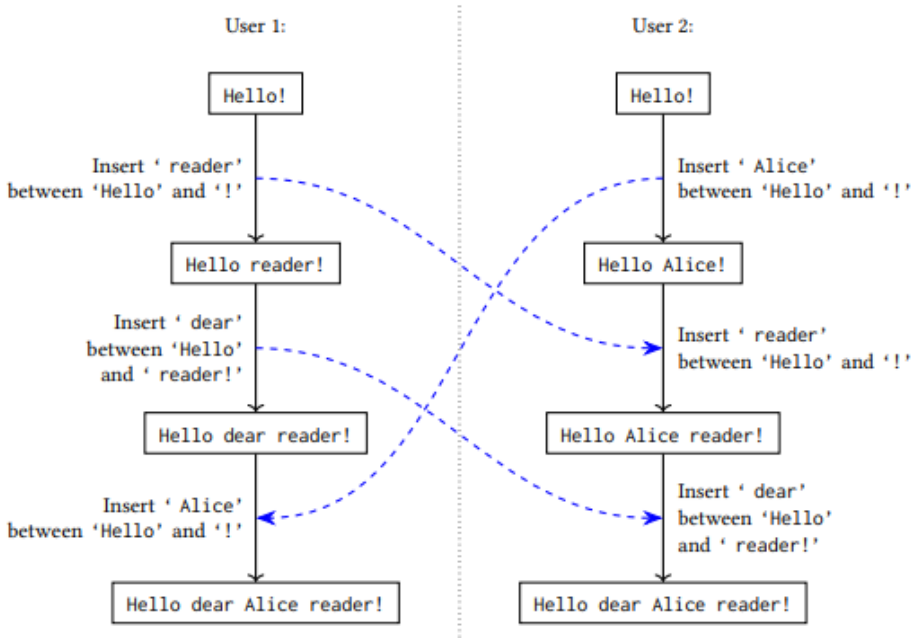


Abbildung 1. Quelle: https://dl.acm.org/doi/pdf/10.1145/3301419.3323972, S. 5

Grund dafür ist die Datenstruktur, welche wie eine Art Baum (engl. tree) (s. Abbildung 2) mit Teil-Bäumen (engl. subtrees) aufgebaut ist. Die Struktur weist nur wenige Verzweigungen aus und jeder Knoten nimmt den Wert eines eingefügten Zeichens an. Jeder übergeordnete Knoten steht für den unmittelbaren Vorgänger eines Zeichens. Die Bearbeitung startet mit dem Knoten bei `head`. Beginnt ein Benutzer das Dokument mit den Zeichen `Hello!`, erscheint jedes weitere Zeichen als Kind in einem Subtree. Das ergibt eine Kette von verknüpften Listen. Wird der Cursor nun zwischen `Hello` und `!` positioniert und Benutzer X tippt `[Leerzeichen]reader`, Benutzer Y tippt `[Leerzeichen]Alice`, dann garantiert RGA durch eine sequentielle Reihenfolge, dass es keine Verschachtelung gibt. In Abbildung 2 ist dargestellt, dass bei jeder Cursorverschiebung ein neuer Subtree erstellt wird – unmittelbar an das Wort `Hello` angrenzend.[25] Bei mehreren Einfügungen an gleicher Position werden sie nach absteigender Reihenfolge der Zeitstempel sortiert. Die Reihenfolge der Einfügungen ist nicht vorhersehbar, weshalb drei mögliche Ergebnisse bei der Zusammenführung entstehen können:

[24] https://dl.acm.org/doi/pdf/10.1145/3301419.3323972, S. 3
[25] https://dl.acm.org/doi/pdf/10.1145/3301419.3323972, S. 4

1. Hello dear Alice reader!
2. Hello Alice dear reader!
3. Hello dear reader Alice!

Es kann festgehalten werden, dass RGA eine Zeichen-für-Zeichen-Verschachtelung ausschließt und daher eine weniger starke Form der Verschachtelung zulässt. Würde ein Benutzer das Dokument in umgekehrter Reihenfolge abtippen – von hinten nach vorne – kann es zu einer willkürlichen Verschachtelung von Zeichen kommen. Grund dafür ist, dass alle Zeichen am `Head` des Dokuments verankert und nach Zeitstempel geordnet werden. Jeder Zeitstempel wird einer Operation zugeordnet und als ID verwendet. [26]

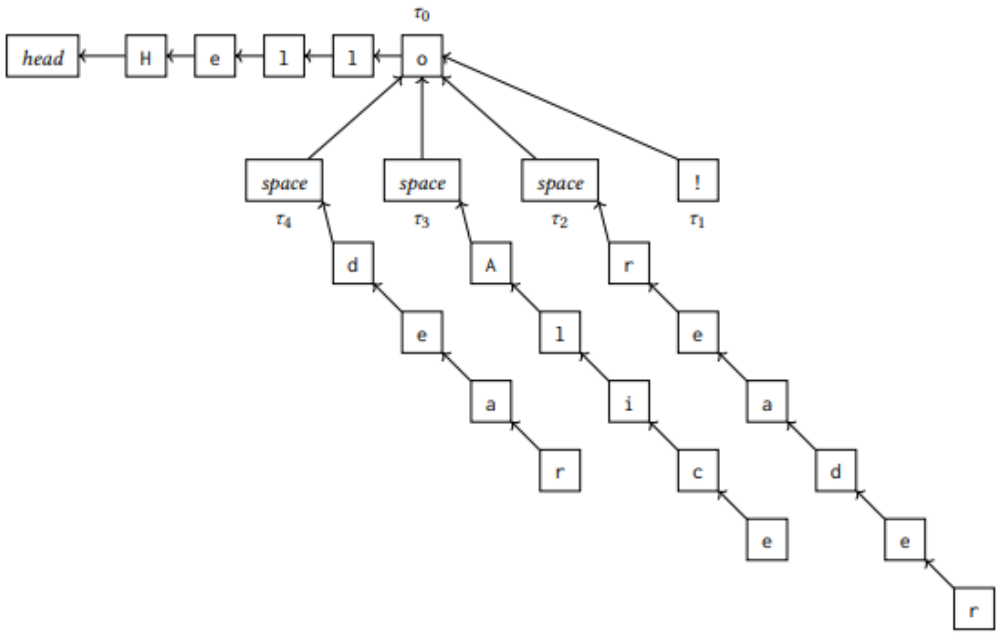


Abbildung 2. Quelle: https://dl.acm.org/doi/pdf/10.1145/3301419.3323972, S. 5

Die Verschachtelungen in RGA können durch eine Zeitstempeldefinition, die als Tripel funktioniert, verbessert werden. Dafür wird das eingefügte Zeichen, der Zeitstempel der Operation und der Zeitstempel des unmittelbaren Vorgängers verwendet. Beim Löschen wird das Zeichen markiert und behält die Position im Dokument, wird jedoch nach dem Löschen ignoriert. Die Autoren Kleppmann et. al. [4] definieren zudem ein weiteres Tupel, welches als Menge der Zeitstempel aller Einfüge-Operationen gilt.[27]

[26] https://dl.acm.org/doi/pdf/10.1145/3301419.3323972, S. 4
[27] https://dl.acm.org/doi/pdf/10.1145/3301419.3323972, S. 4

8. Anwendung von CRDT in Listen

In Listen soll CRDT die Möglichkeit bieten, die Position eines Elements zu verschieben. In einer gegebenen Liste (s. Abb. 3) wird das Ziel verfolgt, das dritte Element an die erste Stelle zu setzen. Alle bereits genannten CRDT-Algorithmen, sind in der Lage diese Operation durchzuführen. CRDTs unterstützen lediglich die Operationen zum Einfügen und Löschen von Elementen. Eine Verschiebung durch das Löschen des Elements an der alten Stelle und das Einfügen an der neuen Stelle kann jedoch zu einer Anomalie führen.[28]

Abbildung 3 zeigt eine gleichzeitige Bearbeitung desselben Eintrags durch zwei Replikate. Dabei wird der Eintrag gelöscht und zwei Mal in das Dokument eingefügt. Dies führt zu Duplikaten, da nun an Position 1 und 2 in der Liste dasselbe Element steht.

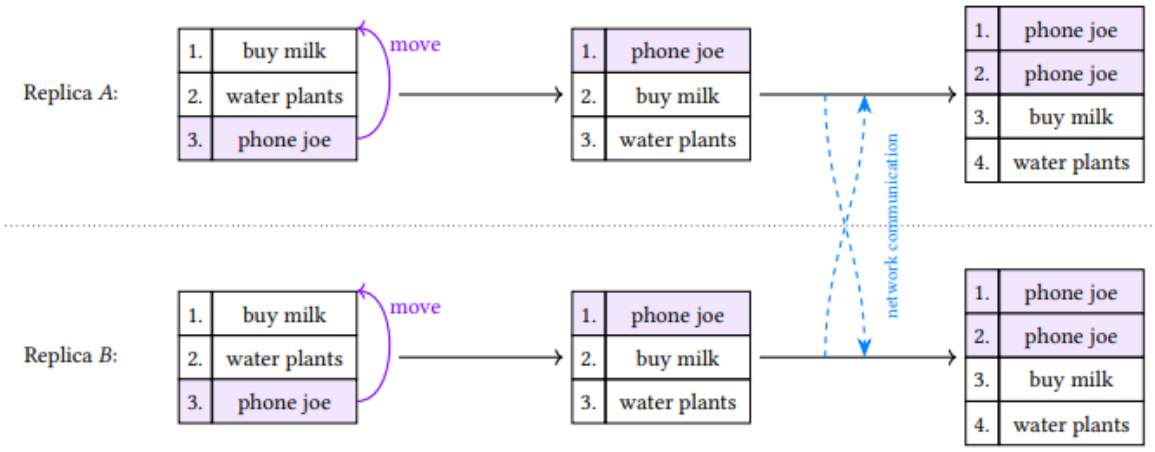


Abbildung 3. Quelle: https://dl.acm.org/doi/pdf/10.1145/3380787.3393677, S. 3

Ein weiteres Problem stellt die gleichzeitige Bearbeitung im Dokument beim Verschieben desselben Elements an verschiedene Positionen dar. Das Element soll nicht dupliziert, sondern nur in einer der beiden Positionen gespeichert werden und durch Warnung signalisieren, dass nicht alle Änderungen umgesetzt wurden. Eine Lösung für das Problem kann durch `Last Writer Wins (LWW) Register` bereitgestellt werden. Ein Register speichert einen an ihn übergebenen Wert und reduziert in diesem Fall alle Aktualisierung auf diesen einen gespeicherten Wert.[29] Beim LWW-Verfahren erhält jede Änderung einen Zeitstempel. Diese stellen sicher, dass die Änderungen

[28] https://martin.kleppmann.com/papers/list-move-papoc20.pdf, S. 1
[29] https://arxiv.org/pdf/1806.10254.pdf, S. 4

eines höheren Zeitstempels die Änderungen mit einem niedrigeren Wert überschreiben. Konvergenz kann hierbei erreicht werden, indem Änderungen eines Replikats ignoriert werden, sollte dieses Replikat eine Änderung mit einem niedrigeren Zeitstempel erhalten als eine andere Änderung.[30]

Damit eine Operation als Gewinner ausgewählt werden kann, registriert eine Variable jeweils einen Wert, der für je einen Benutzer steht. Nach der Zuweisung der Werte kann der Register einen Benutzer als Gewinner auswählen und das Dokument zusammenführen. Die Operation des Verlierers wird ignoriert (s. Abbildung 4).[31]

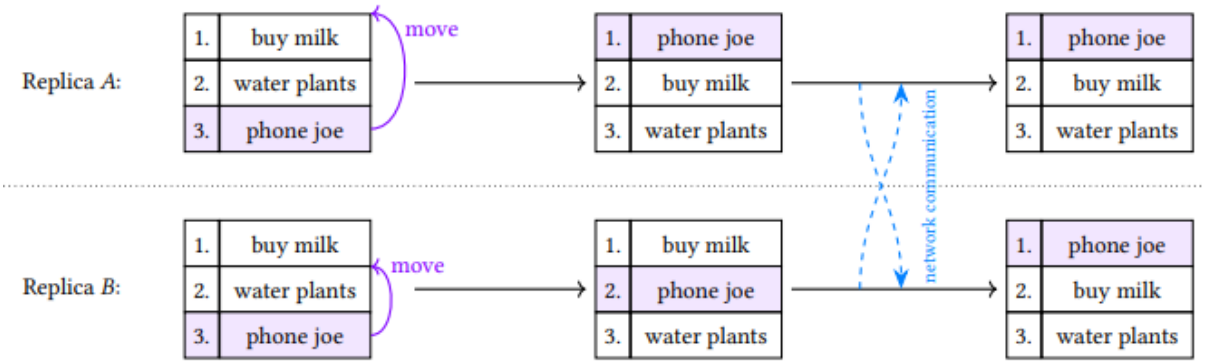


Abbildung 4. Quelle: https://dl.acm.org/doi/pdf/10.1145/3380787.3393677, S. 3

Damit der Algorithmus den richtigen Gewinner auswählen kann, wird ein LWW-Register sowie eine weitere Art der Referenzierung in der Liste benötigt. Unique IDs für Zeichen innerhalb eines Textdokuments sind durch die Algorithmen bereits vorhanden und können für die Position eines Listenelements verwendet werden. Der Algorithmus Treedoc benutzt einen Pfad durch die binäre Baumstruktur, Loogot verwendet einen Pfad mit höherem Verzweigungsfaktor und RGA nutzt den s4vector-Zeitstempel. Als Ergebnis liefern die Algorithmen eine Unique ID, welche in dem LWW-Register eingetragen werden können. Darüber hinaus wird eine weitere Registrierung für die Speicherung der Position eines jeden Listenelements benötigt, ein sogenanntes Add-wins Set (AWSet).[32]

Ein AWSet stellt einen Datentyp für Mengen dar. Eine Einfüge-Operation wird dabei im Gegensatz zu einer Lösch-Operation priorisiert. Wird ein Zeichen von einem Replikat

[30] https://crdt.tech/glossary
[31] https://dl.acm.org/doi/pdf/10.1145/3380787.3393677, S. 2
[32] https://dl.acm.org/doi/pdf/10.1145/3380787.3393677, S. 3

X gelöscht und wieder eingefügt und gleichzeitig von einem Replikat Y gelöscht, so ist das Zeichen bei Zusammenführung ein Element der Menge.[33]
Der Inhalt des AWSets besteht schließlich aus allen Elementen der Liste. Jedes Listenelement besteht aus dem Wert (z. B. aus dem Text des Listenelements) und dem LWW-Register, welches die Position des Listenelements gespeichert hat (s. Abbildug 5). Daraus resultiert, dass neue Elemente zur Liste hinzugefügt werden können, indem sie an das AWSet übergeben werden. Die Position eines Listenelements kann anschließend durch die Aktualisierung des LWW-Register-Werts verschoben werden.

$$\begin{aligned} state = \mathrm{AWSet}(\{&(v_1, \mathrm{LWWRegister}(p_1)), \\ &(v_2, \mathrm{LWWRegister}(p_2)), \ \ldots\}) \end{aligned}$$

Abbildung 5. Quelle: https://dl.acm.org/doi/pdf/10.1145/3380787.3393677, S. 4

Oftmals werden in Listen Drag-&-Drop-Funktionalitäten bereitgestellt und sperren so die Option, mehrere Elemente auf einmal zu verschieben. Im Rahmen von Textbearbeitungen können jedoch auch Listen in Form von reinem Text mit Aufzählungszeichen verwendet werden. Ein Listenelement beginnt in diesem Fall mit dem Aufzählungszeichen und endet mit einer neuen Zeile.[34]
In Abbildung 6 wird ein Beispiel einer Verschiebung von einem Bereich von Elementen und ihr gewünschtes Ergebnis gezeigt. Hier beginnt Replikat B mit dem Verschieben des gesamten Bereichs des Elements `Milk` vor das Element `Bacon`. Gleichzeitig ändert Replikat A das Element `Milk` zu `Soy milk`, indem das großgeschriebene `M` gelöscht wird und die Einfüge-Operation `Soy` sowie ein kleingeschriebenes `m` erfolgt. Das gewünschte Ergebnis liefert die Änderung von `Milk` zu `Soy milk` und setzt das Element `Milk` vor das Element `Bacon`. Demnach steht `Soy milk` an erster Stelle, was bedeutet, dass die Änderungen wie erwartet zusammengeführt wurden.[35]

[33] https://crdt.tech/glossary
[34] https://martin.kleppmann.com/papers/list-move-papoc20.pdf, S. 4
[35] https://martin.kleppmann.com/papers/list-move-papoc20.pdf, S. 4

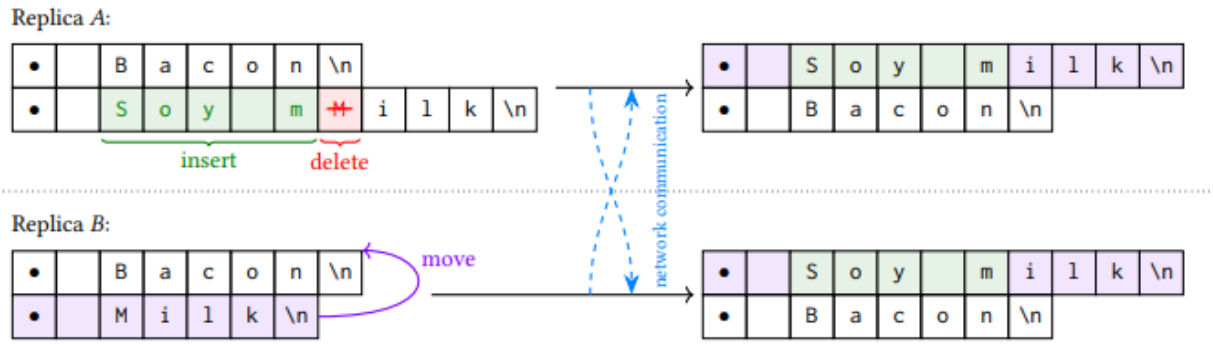


Abbildung 6. Quelle: https://martin.kleppmann.com/papers/list-move-papoc20.pdf, S. 5

Die zuvor betrachteten Algorithmen für Änderungen von einzelnen Elementen führen jedoch zu einem anderen als dem gewünschten Ergebnis. In Abbildung 7 wird dargestellt, dass die Änderung von Replikat B das Element `Milk` vor das Element `Bacon` setzt. Die Änderung von Replikat A tritt allerdings an der alten Position von `Milk` in Kraft. Aus der Ausgangssituation einer Liste mit zwei Zeilen und einem gewünschten Ergebnis bestehend aus zwei Zeilen, wird nun ein Ergebnis bestehend aus drei Zeilen. Dabei steht die letzte Zeile `Soy m` ohne weiteren Kontext an letzter Stelle, da der Kontext in Zwischenzeit die Position geändert hat. Es kann festgehalten werden, dass in der Praxis noch keine Lösung für dieses Problem angeboten wird.[36]

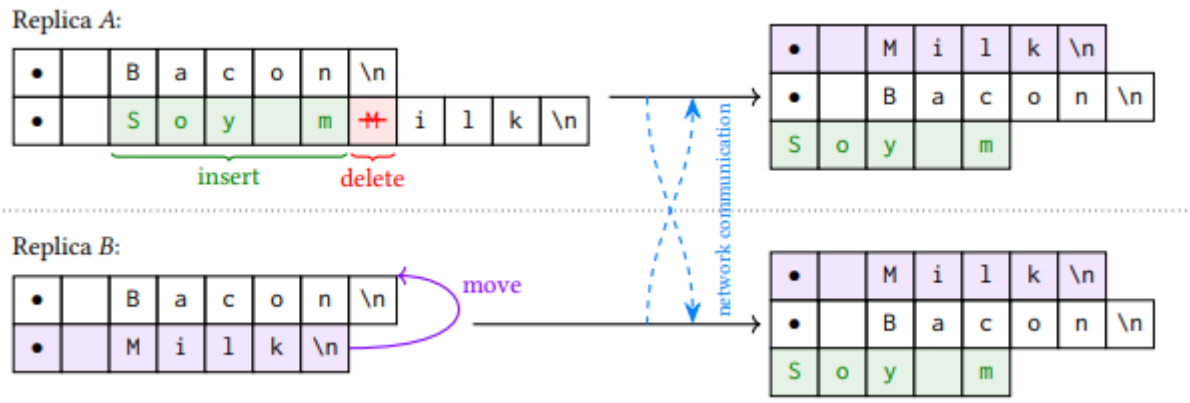


Abbildung 7. Quelle: https://martin.kleppmann.com/papers/list-move-papoc20.pdf, S. 4

[36] https://martin.kleppmann.com/papers/list-move-papoc20.pdf, S. 4

9. CRDTs in Baumstrukturen

Beim Verschieben von Elementen in Listen liefern CRDTs oftmals Anomalien. Dieselben Anomalien können innerhalb von Baumstrukturen ebenfalls auftreten. Die Abbildung 8 stellt eine Baumstruktur dar, welche ein mögliches Dateisystem abbildet. Hierbei stellen die Knoten A, B und C die Kinder von `root` dar. Wird der Knoten A gleichzeitig von zwei Replikaten verschoben, kann das zu unterschiedlichen Ergebnissen nach der Kommunikation der Replikate kommen. Im angeführten Beispiel verschiebt Replikat 1 den Knoten A zu einem Kind von Knoten B. Replikat 2 verschiebt den Knoten A zu einem Kind von Replikat C. Die Ergebnisse können hierbei unterschiedlich ausfallen. In Abbildung 8 zeigt (a), dass das Ergebnis Duplikate liefert. Somit gäbe es Knoten A mit allen weiteren Kindern (a1, a2, a3) als Kind von Knoten B und separat als Kind von Knoten C. Die gleichzeitige Verschiebung hätte zur Folge, dass Änderungen am duplizierten Teilbaum A nur auf einer der Kopien stattfinden. Als Resultat hätten die Benutzer Kopien, die nach weiterer Bearbeitung inkonsistent werden.[37]

Eine weitere Möglichkeit des Ergebnisses wird in (b) gezeigt, welches Knoten A als gemeinsames Kind von B und C darstellt. Diese Variante gilt jedoch nicht als Baumstruktur, da gemäß Definition kein Knoten zwei Eltern hat. Wird als Ergebnis eine Baumstruktur gefordert, kann (b) demnach kein wünschenswertes Ergebnis liefern. Womit (c) und (d) als einzige Ergebnisse ein zufriedenstellendes Produkt bilden. Hierbei wird entweder die Verschiebe-Operation von Replikat 1 oder Replikat 2 ignoriert und liefert somit nur den Teilbaum in einem der beiden Zielverzeichnisse. Es wird, wie zuvor bei den Listen, ein Gewinner in einer Art LWW-Verfahren (durch Zeitstempel) festgelegt.[38]

[37] https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=9563274, S. 1712-1713
[38] https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=9563274, S. 1712-1713

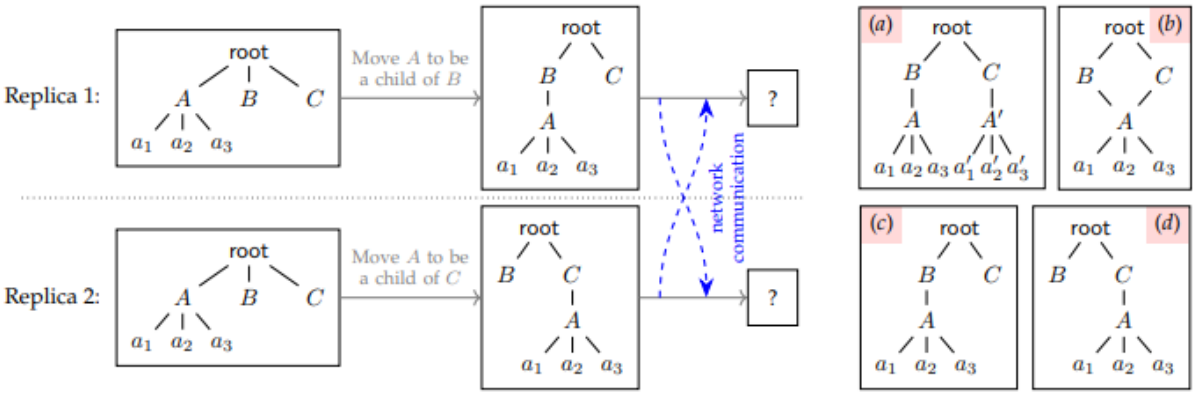


Abbildung 8. Quelle: https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=9563274, S. 1712

Über die bereits genannten Herausforderungen hinaus, weisen Baumstrukturen weitere Schwachstellen auf. In einem Dateisystem sind keine rekursiven Verschiebeoperationen erlaubt. Die Änderung wie in Abbildung 9 (a) dargestellt, ist jedoch rekursiv und stellt somit keine Baumstruktur dar. Die Herausforderung besteht erneut in der replizierten Umgebung, in der gleichzeitige Änderungen erlaubt sind. Mehrere Replikate können Operationen durchführen, welche einzeln den Syntaxanforderungen der Umgebung entsprechen. Die Kombination der Operationen verstößt jedoch gegen die Syntax und ist daher nicht zulässig. In einem weiteren Beispiel, welches in Abbildung 9 dargestellt ist, beginnen beide Replikate mit demselben Baum. Replikat 1 verschiebt Knoten B zu einem Kind von A, wohingegen Replikat 2 Knoten A zu einem Kind von B verschiebt. Das Ergebnis in (a) wird rekursiv und in (b) wird erneut der Knoten A dupliziert. Beide Resultate liefern kein zulässiges Ergebnis. Auch in diesem Beispiel wird das LWW-Verfahren angewendet. Es wird entweder das Ergebnis (d) ignoriert und damit die Operation von Replikat 2 als Gewinner festgelegt oder das Ergebnis (c) wird ignoriert und die Operation von Replikat 1 gewinnt. Eine weitere Herausforderung ist in diesem Zusammenhang die Wahl des Gewinners. Es ist eine von zwei oder mehr kollidierender Operationen auszuwählen, sodass am Ende alle Replikate konvergent sind.[39]

[39] https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=9563274, S. 1712-1713

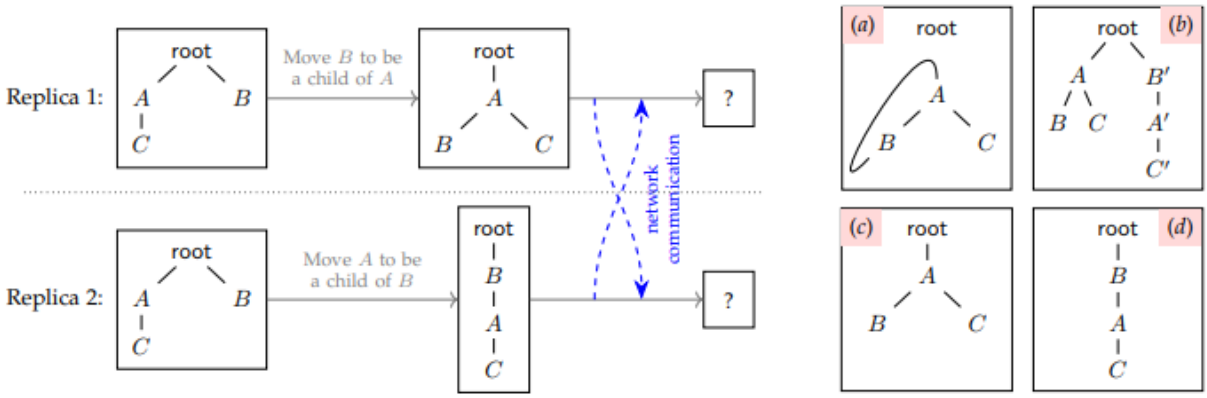


Abbildung 9. Quelle: https://martin.kleppmann.com/papers/move-op.pdf, S. 3

Jede Operation kann durch Zeitstempel in Sequenzen wiedergegeben werden. Mithilfe der Zeitstempel können diese Sequenzen die Operationen von zwei Benutzern verschmelzen. Voraussetzung hierfür ist, dass die Kommunikation im Rahmen einer replizierten Umgebung stattfindet und diese nicht über einen zentralen Server läuft. Änderungen von Benutzern werden auf den lokalen Replikaten angewendet und asynchron über das Netzwerk übertragen. Es gibt drei Operationen, die eine Baumstruktur ändern können. Diese sind das Einfügen, das Verschieben und das Löschen eines Knotens. Alle drei Änderungen können jedoch auch durch die Verschiebe-Operation allein umgesetzt werden.[40]

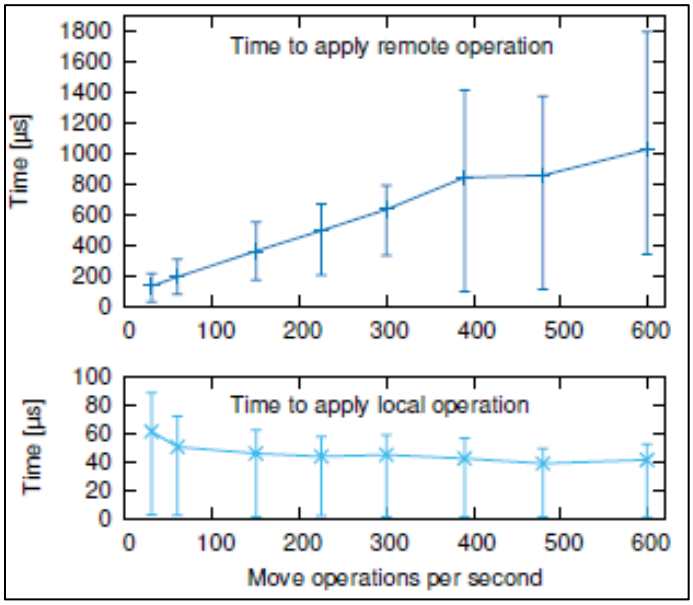
Ziel ist es, die Sequenzen der Operationen so zusammenzuführen, dass eine einzige Sequenz von Operationen in der Reihenfolge der Zeitstempel entsteht. Eine optimistische Replikation erlaubt den Replikaten die Durchführung von Operationen in beliebiger Reihenfolge. Folglich muss sichergestellt werden, dass die Baumstruktur am Ende für jedes Replikat zu einem konsistenten Zustand konvergiert. Hierfür sind drei Funktionen notwendig, welche in der Sprache Isabelle/HOF mit `undo op`, `redo op`, `apply op` aufgerufen werden können.[41] Zunächst werden alle Operationen mit einem Zeitstempel der größer ist als die gewünschte Einfüge-Operation rückgängig gemacht (per `undo op`). Die gewünschte Einfüge-Operation wird anschließend durch die Funktion `apply op` in die Operations-Reihenfolge eingegliedert. Im letzten Schritt werden alle rückgängig gemachten Operationen durch `redo op` wieder hinzugefügt. Das Resultat besteht darin, dass alle Operationen den Anschein erwecken, dass diese

[40] Quelle: https://martin.kleppmann.com/papers/move-op.pdf, S. 4
[41] https://martin.kleppmann.com/papers/move-op.pdf, S. 6 (Zeilen 32-49)

nach aufsteigendem Zeitstempel getätigt wurden. Im Hintergrund können die Operationen jedoch in einer beliebigen Reihenfolge getätigt worden sein.[42]

10. Performance von CRDT-Algorithmen

Werden Operation auf die zuvor beschriebene Logik angewendet, entsteht eine große Anzahl von `undo`- und `redo`-Operationen. Es müssen demnach viele Operationen rückgängig gemacht und wieder eingefügt werden. Die Abbildung 10 zeigt ein Experiment, in dem drei Replikate auf drei verschiedenen Kontinenten Operationen tätigen. Die Replikate sind in Kalifornien (us-west-1), Irland (eu-west-1) und Singapur (ap-southeast-1) lokalisiert. Die Netzwerkverzögerungen zwischen Irland und Kalifornien betragen durchschnittlich 145 ms. Der Weg von Singapur nach Kalifornien dauert 176 ms. Das Diagramm `Time to apple remote operation` zeigt die Verteilung der Zeiten für die Anwendung einer Operation, welche von einem anderen Replikat empfangen wurde. Das Diagramm `Time to apply local operation` zeigt die Zeit für die Anwendung einer lokal erzeugten Operation. Die Anzahl der Verschiebevorgänge steht in Zusammenhang mit den `undo`- und `redo`-Operationen. Je größer die Anzahl der Vorgänge, desto langsamer wird die Ausführung einer Operation. Die Kosten einer Verschiebeoperation werden mit O(nd) berechnet. Dabei steht `n` für die Anzahl der `undo`- und `redo`-Operationen, welche im Log-Protokoll gespeichert sind und `d` spiegelt die Anzahl der übergeordneten Beziehungen im Baum wider. Ein Replikat wendet eine Operation zunächst lokal an und sendet sie asynchron an die beiden anderen Replikate. Es kann gesagt werden, dass jedes Replikat etwa ein Drittel der Operationen lokal erzeugt und zwei Drittel von den anderen Replikaten empfängt. Das



Abbidlung 10. Quelle: https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=9563274, S. 1719

[42] https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=9563274, S. 1714 u. 1716

Ergebnis des Experiments ist, dass 600 Operationen/Sek. durchgeführt werden können.[43]

11. Zusammenfassung

CRDTs ermöglichen die Replikation von Datenstrukturen ohne zentrale Kommunikation über einen Server. Dies ermöglicht eine dezentralisierte und verteilte Datenreplikation. Allerdings können Konflikte auftreten, wenn mehrere Benutzer gleichzeitig an verschiedenen Replikaten arbeiten und diese aktualisieren. Bei den meisten hier vorgestellten Algorithmen für die Textbearbeitung treten Verschachtelungen und Anomalien auf. Diese können die korrekte Zusammenführung der Daten beeinträchtigen und es erschweren, ein sinnvolles bzw. wünschenswertes Ergebnis für den Menschen zu erstellen.

Die Herausforderungen bei CRDT-Algorithmen bestehen darin, sie für komplexe Datenstrukturen wie Listen und Bäume zu implementieren. Die Entwicklung eines gut konzipierten CRDT-Algorithmus stellt für deren Designer eine komplexe Aufgabe dar. Ein weiteres Problem besteht bei Verschiebe-Operationen, für die bisher teilweise noch keine anerkannte Lösung gefunden wurde. Zusätzlich müssen CRDT-Algorithmen effizient gestaltet werden, um eine gute Systemperformance zu gewährleisten.

[43] https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=9563274, S. 1720

Quellen

[1]	CRDT-Tech (a)	About CRDTs https://crdt.tech/
[2]	CRDT-Tech (b)	CRDT Glossary https://crdt.tech/glossary
[3]	Kleppmann	Research for Practice: Convergence https://martin.kleppmann.com/papers/convergence-cacm.pdf
[4]	Kleppmann et. al.	Interleaving anomalies in collaborative text editors https://dl.acm.org/doi/pdf/10.1145/3301419.3323972
[5]	Kleppmann et. al.	Moving Elements in List CRDTs https://martin.kleppmann.com/papers/list-move-papoc20.pdf
[6]	Kleppmann et. al.	A Highly-Available Move Operation for Replicated Trees https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=9563274
[7]	Preguiça	Conflict-free Replicated Data Types: An Overview https://arxiv.org/pdf/1806.10254.pdf
[8]	Shapiro et. al.	Conflict-free Replicated Data Types https://pages.lip6.fr/Marek.Zawirski/papers/RR-7687.pdf
[9]	Sun et. al.	Real Differences between OT and CRDT in Correctness and Complexity for Consistency Maintenance in Co-Editors https://dl.acm.org/doi/pdf/10.1145/3392825
[10]	Sypytkowski	An introduction to state-based CRDTs https://www.bartoszsypytkowski.com/the-state-of-a-state-based-crdts/
[11]	Weidner et. al.	The Art of the Fugue: Minimizing Interleaving in Collaborative Text Editing https://arxiv.org/pdf/2305.00583.pdf